

공학 학사 학위논문

뇌-컴퓨터 인터페이스의 실시간 연산을 위한
스파이크 분류 알고리즘 최적화

2026년 2월

서울대학교 공과대학

전기·정보공학부

김 해 원

뇌-컴퓨터 인터페이스의 실시간 연산을 위한
스파이크 분류 알고리즘 최적화

지도교수 김장우

이 논문을 공학 학사학위 논문으로 제출함.

서울대학교 공과대학

전기·정보공학부

김 해 원

김해원의 학사 학위 논문을 인준함.

2025년 월 일

지도교수

(인)

초 록

본 연구는 실시간 뇌-컴퓨터 인터페이스(Brain-computer interface, BCI) 응용에서 필수적인 스파이크 분류 알고리즘의 처리 지연과 메모리 제약 문제를 해결하고자 수행되었다. 전극으로부터 수집되는 다채널 신경 신호는 여러 뉴런의 발화가 중첩된 형태로 기록되므로, 개별 뉴런의 발화 시점과 소속을 식별하기 위한 스파이크 분류 과정이 필요하다. 특히 BCI나 폐쇄형 피드백 시스템에서는 분류 결과가 실시간으로 제공되어야 하며, 임베디드 환경에서는 연산 자원과 메모리가 제한적이기 때문에 연산을 효율적으로 수행할 필요성이 강조된다.

이에 본 논문에서는 템플릿 기반 스파이크 분류 알고리즘인 SpyKING CIRCUS를 대상으로 병렬화 및 코드 수준의 최적화를 적용하여 처리 성능을 향상시켰다. 구체적으로는 파이썬 기반의 병렬 통신 라이브러리인 mpi4py를 활용해 채널 단위 분산 처리를 도입하였고, 반복적인 연산을 벡터화로 반복문을 제거하였다. 또한 루프 내 불필요한 메모리 복사를 제거하고, 클러스터 관리 단계의 자료 구조와 연산 순서를 개선하여 메모리 할당 오버헤드를 낮추었다.

평가에는 SpikeForest의 hybrid 데이터셋(샘플링 주파수 30 kHz, 64채널)을 사용하였고, 동일한 입력과 처리 조건에서 최적화 전후의 처리 시간을 비교하였다. 그 결과, 최적화 전 전체 처리 시간은 516.0초였으나, 제안한 병렬화와 모든 코드 최적화를 적용한 후에는 238.7초로 단축되어 약 53.7%의 총 처리 시간 향상을 달성하였다. 아울러 클러스터의 수와 각 클러스터에 할당된 스파이크 개수는 최적화 전후에 차이가 없어, 처리 속도 향상에도 불구하고 분류 정확도에는 유의미한 손실이 없음을 확인하였다.

본 연구의 기여는 두 가지로 정리된다. 첫째, 정밀한 프로파일링에 기반하여 어떤 연산이 왜 비용을 유발하는가를 규명하고, 그 결과를 토대로 반복문·메모리 접근·자료구조 수준에서 실질적 오버헤드를 제거하는 실용적 개선 방법을 제시한 점이다. 둘째, 구현 수준의 최적화가 온라인 스파이크 분류의 처리 성능을 크게 개선할 수 있음을 실험적으로 입증하고, 병렬 확장 시 통신 비용이 성능에 미치는 영향을 계량적으로 분석한 점이다. 다만 병렬 확장 과정에서 통신 오버헤드에 의해 성능이 역전되는 구간이 관찰되었고, 일부 내부 연산에 잔여 병목이 존재함을 확인하였다. 이러한 한계는 향후 해결해야 할 과제로 남아 있다.

결론적으로 본 연구는 알고리즘적 아이디어의 보존 하에 구현·데이터 접근·연산 흐름을 개선함으로써 온라인 스파이크 분류의 실시간 처리 능력을 의미있게 향상시켰다. 이는 실시간 BCI 시스템의 응답성을 높이고, 제한된 자원 환경에서의 적용 가능성을 확대하는 데 기여할 것으로 기대된다.

주요어 : 뇌-컴퓨터 인터페이스, 스파이크 분류, 병렬 컴퓨팅, 실시간 처리

목 차

초 록	i
목 차	ii
그림 목차	iii
1 서론	1
1.1 뇌-컴퓨터 인터페이스	1
1.2 SpyKING CIRCUS 개요	2
1.3 연구의 목적 및 해결 과제	3
2 본론	5
2.1 알고리즘의 전체 흐름	5
2.2 MPI 기반 채널 병렬화	7
2.3 Pruning 최적화	8
2.4 메모리 최적화	9
2.5 연산 최적화	10
2.5.1 피크 검출 연산 최적화	10
2.5.2 템플릿 거리 연산 최적화	11
3 결론	12
3.1 결과 분석	12
3.2 기여 및 한계	13
3.3 향후 연구 방향	14
참고 문헌	15
Abstract	16

그림 목차

1.1	뇌-컴퓨터 인터페이스 구조	2
1.2	SpyKING CIRCUS 스파이크 분류 알고리즘의 전체 흐름	2
2.1	병렬화로 개선된 온라인 스파이크 처리 흐름	5
2.2	단일 프로세스와 MPI 병렬 처리 파이프라인 비교	7
2.3	메모리 복사 오버헤드 제거를 위한 리스트 운용 구조 개선	8
2.4	사전 연산 기반의 메모리 접근 구조 개선	9
2.5	반복문 제거를 통한 병렬적 피크 검출 구조	10
2.6	행렬 연산을 통한 다중 템플릿 거리 일괄 계산	11
3.1	최적화 기법의 순차적 적용에 따른 처리 시간 변화	12
3.2	병렬화에 따른 처리 시간 변화	13

제 1 장

서론

1.1 뇌-컴퓨터 인터페이스

뇌-컴퓨터 인터페이스(Brain-Computer Interface, BCI)는 뇌에서 발생하는 신경 신호를 실시간으로 측정하고 해석하여, 근육이나 말초 신경을 거치지 않고 외부 장치를 직접 제어하거나 컴퓨터와 통신할 수 있게 하는 기술이다. 그림 1.1은 이러한 BCI 시스템의 전반적인 구조와 동작 원리를 보여준다. 이 시스템은 크게 뇌 신호의 획득, 신호 처리, 그리고 응용의 세 단계로 구성된다.

우선, 뇌 내부나 표면에 부착된 전극을 통해 신경 세포(Neuron)들의 전기적 활동을 감지한다. 이렇게 획득된 아날로그 신호는 증폭 및 디지털 변환 과정을 거쳐 프로세서로 전달된다. 프로세서 내부에서는 전 처리(Preprocessing), 스파이크 탐지(Spike Detection) 및 분류(Spike Sorting), 그리고 디코딩(Decoding) 알고리즘이 수행된다. 이 과정을 통해 사용자의 운동 의도나 뇌 상태가 해석되며, 최종적으로 로봇 팔과 같은 보철 기기를 제어하거나, 그림 1.1의 우측 예시와 같이 뇌전증 발작을 감지하여 전기 자극을 통해 떨림을 억제하는 등의 치료적 개입을 수행하게 된다.

BCI 기술은 사지 마비 환자나 루게릭병 환자의 운동 및 의사소통 기능을 복원하는 데 핵심적인 역할을 한다 [1]. 특히 최근에는 단순히 신호를 읽는 것을 넘어, 뇌에 적절한 자극을 되돌려주는 폐쇄형(Closed-loop) BCI 시스템이 주목받고 있다. 이러한 고성능 BCI 시스템을 구현하기 위해서는 다채널 신경 신호에서 혼합된 파형을 분석하여 개별 뉴런의 활동을 분리해내는 스파이크 분류(Spike Sorting) 과정이 필수적이다 [2]. 대표적으로 템플릿 매칭 기반의 알고리즘인 SpyKING CIRCUS [3]는 대규모 채널 데이터에 대해 높은 정확도를 보이지만, 연산 복잡도가 높아 실시간 시스템에 적용하기 위해서는 구조적인 개선이 요구된다.

더욱이 BCI 시스템의 궁극적인 목표인 완전 이식형 장치를 구현하기 위해서는 하드웨어의 물리적 제약을 고려해야 한다. 뇌 조직은 열에 매우 민감하여 1°C의 온도 상승만으로도 영구적인 손상을 입을 수 있으므로 [4], 이식형 칩의 전력 소모와 발열을 엄격히 제한해야 한다. 이를 해결하기 위해 HALO [5],

SCALO [6]와 같은 저전력 하드웨어 가속기들이 활발히 연구되고 있으나, 제한된 하드웨어 자원 내에서 수천 채널의 데이터를 실시간으로 처리하기 위해서는 하드웨어뿐만 아니라 알고리즘 레벨에서의 극한의 최적화가 동반되어야 한다. 따라서 본 연구에서는 SpyKING CIRCUS 알고리즘을 기반으로, 병렬 처리 및 코드 최적화를 통해 분류 정확도를 유지하면서도 처리 속도를 획기적으로 개선하여 실시간 BCI 시스템에 적용 가능한 방안을 제안하고자 한다.

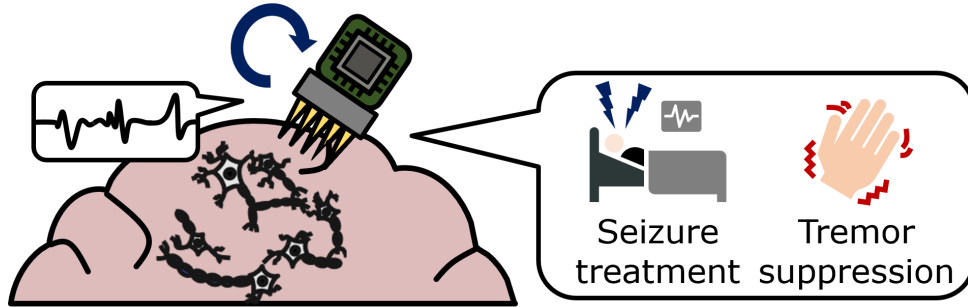


그림 1.1: 뇌-컴퓨터 인터페이스 구조

Figure 1.1: Overall architecture of a Brain-computer interface

1.2 SpyKING CIRCUS 개요

SpyKING CIRCUS는 수천 개 이상의 채널을 가진 고밀도 전극에서 수집된 대규모 신경 신호를 분석하기 위해 개발된 알고리즘이다 [3]. 이 알고리즘은 사전에 정의된 파형 모델인 템플릿(Template)과 입력 신호를 대조하여 스파이크를 분류하는 방식을 채택하고 있다. 단순한 임계값 기반 분류와 달리, 템플릿 대조 방식은 잡음이 심하거나 여러 뉴런의 신호가 겹쳐서 나타나는 상황에서도 우수한 분리 성능을 보인다. 그림 1.2는 본 연구의 기반이 되는 SpyKING CIRCUS의 전체 동작 흐름을 보여준다.

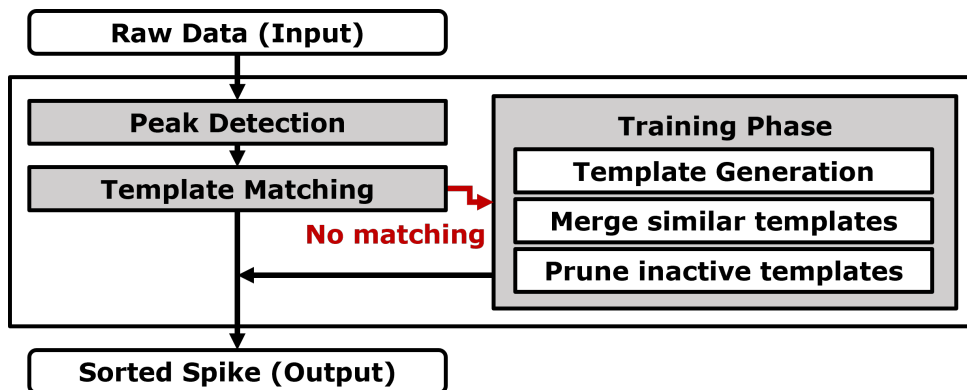


그림 1.2: SpyKING CIRCUS 스파이크 분류 알고리즘의 전체 흐름

Figure 1.2: Overall pipeline of SpyKING CIRCUS spike sorting algorithm

데이터 처리 과정은 크게 스파이크 탐지와 템플릿 매칭의 두 단계로 진행된다. 먼저, 입력되는 원시 데이터에서 잡음을 제외하고 유의미한 전압 변화가 발생하는 구간을 찾아낸다. 이렇게 검출된 파형은 기존에 학습된 템플릿 사전(Dictionary)과 비교되며, 가장 유사한 형태를 가진 템플릿의 뉴런으로 즉시 분류된다. 이 방식은 복잡한 연산을 미리 계산된 템플릿과의 비교 연산으로 대체함으로써, 대용량 데이터에서도 비교적 빠른 처리 속도를 보장한다.

특히 이 알고리즘의 가장 큰 장점은 그림 1.2의 우측에 나타난 학습 과정을 통해 변화하는 신경 신호 환경에 능동적으로 대처할 수 있다는 점이다. 만약 입력된 파형이 기존의 어떤 템플릿과도 일치하지 않는다면, 알고리즘은 이를 새로운 뉴런의 활동으로 간주하여 신규 템플릿을 생성한다. 또한, 시간이 지남에 따라 전극의 미세한 움직임 등으로 인해 파형의 모양이 변할 수 있는데, 이를 보정하기 위해 유사해진 템플릿들을 하나로 병합하거나 오랫동안 활동이 없는 템플릿은 메모리에서 제거하는 최적화 과정을 수행한다. 이러한 적응형 구조는 장시간 동작해야 하는 BCI 시스템에서 신호의 비정상성(Non-stationarity)을 극복하는 데 필수적이다.

하지만 기존의 SpyKING CIRCUS 구현체는 대용량 데이터를 디스크에 저장한 뒤 순차적으로 분석하는 오프라인 처리를 전제로 설계되어 있어, 즉각적인 피드백이 요구되는 최신 BCI 트렌드에 부합하기에는 연산 속도 면에서 한계가 있다. 본 연구는 이러한 오프라인 기반의 처리 구조와 파이썬 언어 환경을 유지하되, 그 안에서 연산 효율을 극대화하여 실시간 처리에 준하는 속도를 확보하는 것을 목표로 한다. 이를 위해 파이썬 언어 환경 내에서의 비효율을 제거하는 코드 레벨의 리팩토링을 수행하고, MPI(Message Passing Interface) 기반의 분산 처리를 도입하여 다수의 연산 노드가 데이터를 병렬로 처리하도록 개선하였다. 즉, 기존 알고리즘의 정확도와 유연성은 그대로 계승하면서, 하드웨어 자원을 효율적으로 활용하는 최적화를 통해 차세대 온라인 BCI 시스템에서도 활용 가능한 고속 스파이크 분류의 가능성을 제시하고자 한다.

1.3 연구의 목적 및 해결 과제

앞선 절에서 기술한 바와 같이 SpyKING CIRCUS는 높은 정확도를 보장하지만, 기본적으로 모든 데이터를 확보한 후 일괄 처리하는 오프라인 분석을 전제로 설계되었다. 따라서 데이터가 실시간으로 입력되는 환경에서 이를 그대로 사용할 경우, 실시간성을 저해하는 구조적인 문제와 연산 비효율성이 발생한다. 본 연구는 SpyKING CIRCUS의 알고리즘적 정확도는 유지하면서, 코드 수준의 최적화와 구조 개선을 통해 연산 처리 시간을 획기적으로 단축하는 것을 목적으로 한다. 이를 위해 본 논문에서는 다음과 같은 구체적인 문제점들을 해결하고자 한다.

첫째, 실시간 스트리밍 처리에 부적합한 연산 구조의 개선이다. 기존 코드는 전체 데이터셋에 대한 반복적인 접근과 연산을 허용하는 구조로 작성되어 있어, 짧은 시간 단위로 데이터가 입력되는 스트리밍 환경에서는 불필요한 연산 오버헤드가 누적된다. 특히 매 반복마다 발생하는 중복된 메모리 할당이나 비효율적인 조건 검사 로직은 전체 처리 시간을 증가시키는 주된 요인이 된다. 따라서 이러한 비효율적인

구현을 찾아내어 연산의 중복을 제거하고, 메모리 접근 패턴을 단순화하여 처리 지연을 최소화해야 한다.

둘째, 순차적 처리 방식의 한계 극복을 위한 병렬화이다. 다채널 신경 신호 데이터는 각 채널이 독립적인 정보를 담고 있는 경우가 많음에도 불구하고, 기존의 단일 프로세스 기반 구현은 이를 순차적으로 처리함으로써 가용한 컴퓨팅 자원을 충분히 활용하지 못하고 있다. 채널 수가 증가할수록 처리 시간이 선형적으로 증가하는 문제를 해결하기 위해, 연산 부하를 다수의 프로세스로 분산시키는 병렬 처리 기법의 도입이 필요하다.

셋째, 알고리즘 구현 최적화를 통한 가속화이다. 복잡한 수치 연산이 포함된 피크 검출 및 템플릿 매칭 과정을 단순한 반복문으로 처리할 경우, 데이터의 양이 늘어남에 따라 연산 속도가 급격히 저하된다. 이를 해결하기 위해 반복적인 스칼라 연산을 벡터 및 행렬 연산으로 대체하여, 동일한 파이썬 환경 내에서도 연산 효율을 극대화할 수 있는 형태로 코드를 재구성해야 한다.

따라서 본 연구는 SpyKING CIRCUS를 기반으로 하되, 실시간 BCI 시스템의 요구사항에 맞춰 코드 리팩토링과 병렬화를 수행함으로써, 데이터 처리의 정확도를 유지함과 동시에 전체 연산 시간을 획기적으로 단축하는 데 초점을 맞춘다.

제 2 장

본론

2.1 알고리즘의 전체 흐름

본 연구에서 제안하는 고속 스파이크 분류 시스템의 전체 프레임워크는 그림 2.1과 같다. 이 그림은 기존의 SpyKING CIRCUS를 그대로 사용한 베이스라인(Baseline) 모델과, 실시간 처리를 위해 본 연구에서 재설계한 가속(Accelerated) 모델의 파이프라인을 비교하여 보여준다.

좌측의 베이스라인 모델은 전처리(Preprocessing), 피크 검출(Peak Detection), 클러스터링 및 매칭(Clustering & Matching), 그리고 템플릿 정리(Pruning / Merging)의 모든 과정이 단일 프로세스(Single-process) 내에서 순차적으로 수행되는 구조를 가진다. 이러한 직렬 구조는 이전 단계의 연산이 완료되어야만 다음 단계로 진행할 수 있으므로, 대용량 신경 신호가 지속적으로 입력되는 환경에서는 연산 지연이 누적되어 실시간성을 저해하는 주된 원인이 된다.

반면, 우측의 제안하는 가속 모델은 이러한 병목을 해소하기 위해 전체 파이프라인을 병렬 처리 구조로 개편하고, 각 단계별로 특화된 최적화 기법을 적용하였다. 전체 흐름은 크게 데이터를 준비하여 분배하는 전처리 단계와, 분배된 데이터를 실제로 분석하는 병렬 연산 단계로 구분된다.

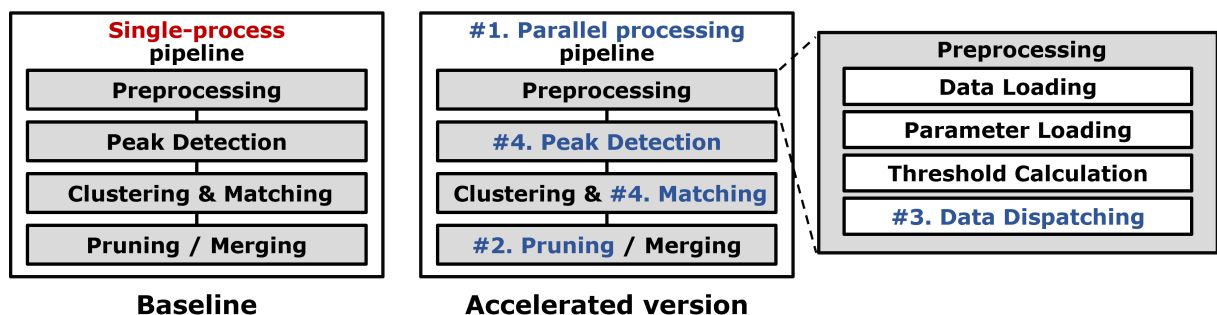


그림 2.1: 병렬화로 개선된 온라인 스파이크 처리 흐름

Figure 2.1: Overall framework of the proposed system

Algorithm 1 Proposed Parallel Spike Sorting Algorithm

Require: Raw neural data stream D , Thresholds θ , Templates T

Ensure: Sorted spikes S

```
1: Initialize MPI environment and load parameters
2: Distribute channels to  $P$  processors ( $N_{ch}/P$  per rank)
3: while stream is active do
4:   Receive data chunk  $X_{local}$  (Time-series data)
5:   Pre-computation:  $X_{neg} \leftarrow -X_{local}$  ▷ Sec. 2.4
6:    $M_{peak} \leftarrow \text{VECTORIZEDPEAKDETECTION}(X_{neg}, \theta)$  ▷ Sec. 2.5.1
7:   for each detected peak  $p$  in  $M_{peak}$  do
8:      $d \leftarrow \text{VECTORIZEDDISTANCE}(p, T)$  ▷ Sec. 2.5.2
9:     Assign cluster ID based on  $\min(d)$ 
10:  end for
11:  if Pruning interval reached then
12:     $\text{INPLACEPRUNING}(T)$  ▷ Sec. 2.3
13:  end if
14:  Barrier Synchronization
15: end while
```

먼저 전처리 단계에서는 데이터를 메모리에 로드(Data Loading)하고 필요한 파라미터와 임계값(Threshold)을 계산한다. 이후 데이터 분배(Data Dispatching, #3) 과정을 통해 전체 데이터를 다수의 연산 노드로 분할하여 전송한다. 데이터를 수신한 각 프로세스는 독립적으로 할당된 채널에 대해 피크 검출(#4)과 템플릿 매칭(#4)을 수행하며, Pruning 및 Merging(#2) 로직을 통해 클러스터 정보를 갱신한다. 이 모든 과정은 MPI(Message Passing Interface)를 기반으로 한 병렬 처리(#1) 환경 위에서 동작한다.

구체적으로 데이터 분배 과정에서는 공간적(Spatial) 분할과 시간적(Temporal) 분할이 동시에 이루어진다. 전체 채널의 데이터는 프로세스 개수에 맞추어 그룹(Batch) 단위로 공간적으로 분할되며, 각 프로세스는 할당받은 채널 그룹에 대해서만 연산을 수행한다. 또한, 긴 시간 동안 지속되는 신경 신호의 특성을 고려하여 데이터는 일정 시간 단위의 청크(Chunk)로 잘려 순차적으로 각 프로세스에 스트리밍된다. 이러한 구조는 전체 데이터를 한 번에 메모리에 올리는 부담을 줄이고, 각 프로세스가 독립적인 캐시(Cache)를 효율적으로 활용할 수 있게 한다.

본 장의 이어지는 절에서는 이 프레임워크를 구성하는 4가지 핵심 최적화 기법에 대해 상세히 기술한다. 2.2절에서는 전체 작업 부하를 분산시키는 병렬 처리 구조를, 2.3절에서는 불필요한 연산을 제거하는 Pruning 최적화를 다룬다. 이어지는 2.4절과 2.5절에서는 각각 메모리 접근 효율을 높이는 데이터 처리 방식과, 주요 병목 구간인 피크 검출 및 거리 계산을 가속화하는 연산 벡터화 기법에 대해 설명한다.

본 연구에서 제안하는 병렬 및 최적화 기법이 통합된 전체 알고리즘의 동작 과정은 알고리즘 1와 같이 요약할 수 있다.

2.2 MPI 기반 채널 병렬화

기존의 단일 프로세스(Single-process) 기반 스파이크 분류 시스템은 그림 2.2의 상단 도식과 같이 모든 채널의 데이터를 순차적으로 처리해야 하는 구조적 한계를 지닌다. 채널 수가 N 개일 때 전체 처리 시간은 $O(N)$ 으로 비례하여 증가하므로, 수천 채널 이상의 고밀도 전극 데이터를 실시간으로 처리하기에는 막대한 지연 시간(Latency)이 발생한다. 이를 해결하기 위해 본 연구에서는 MPI(Message Passing Interface)를 도입하여 연산 부하를 다수의 프로세서에 분산시키는 채널 기반 병렬화(Channel-based Parallelization) 구조를 구현하였다.

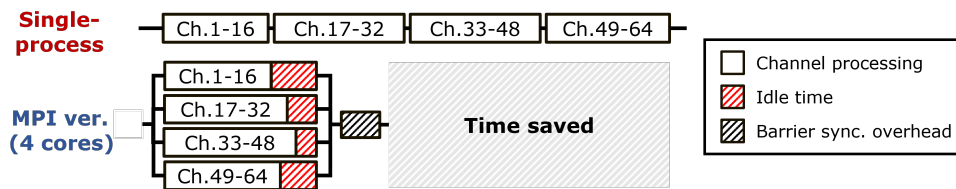


그림 2.2: 단일 프로세스와 MPI 병렬 처리 파이프라인 비교

Figure 2.2: Comparison of single-process and MPI parallel processing pipelines

제안하는 병렬 처리 구조는 공간적 분할(Spatial Partitioning)과 시간적 분할(Temporal Partitioning)을 결합하여 동작한다. 먼저 공간적 측면에서, 전체 전극 채널은 가용한 프로세서(Core)의 개수에 맞추어 균등하게 분배된다. 예를 들어 64채널 데이터를 4개의 프로세서로 처리할 경우, 그림 2.2의 하단 예시와 같이 각 프로세서는 16개의 채널(Ch.1-16, Ch.17-32 등)로 구성된 로컬 배치(Local Batch)를 할당받는다. 각 프로세서는 할당된 채널에 대해서만 독립적으로 피크 검출 및 템플릿 매칭을 수행하므로, 이론적으로 전체 연산 속도는 프로세서 개수에 비례하여 향상된다. 시간적 측면에서는 연속적인 신경 신호 스트림을 일정 시간 단위(예: 60,000 샘플)의 청크(Chunk)로 분할하여 처리한다. 이때 청크의 크기는 시스템의 전체 지연 시간과 처리량 사이의 트레이드오프를 결정하는 중요한 파라미터이다. 청크 크기가 너무 작으면 잦은 MPI 통신과 동기화 오버헤드로 인해 처리 효율이 저하되는 반면, 청크가 너무 크면 데이터가 쌓일 때까지 기다려야 하므로 입력-출력 간의 지연 시간이 증가하여 실시간 피드백이 어려워진다. 본 연구에서는 이러한 특성을 고려하여, 통신 오버헤드를 최소화하면서도 BCI 애플리케이션이 요구하는 응답 속도(예: 100ms 이내)를 만족시킬 수 있는 최적의 청크 크기를 실험적으로 선정하여 적용하였다.

각 프로세서는 자신의 채널에 해당하는 데이터 청크를 처리한 후, mpi4py 라이브러리의 통신 기능을 통해 결과를 집계하고 동기화(Synchronization) 과정을 거친다. 이때 모든 프로세서가 현재 청크의 처리를 완료할 때까지 대기하는 배리어(Barrier) 동기화가 이루어지며, 이후 다음 시간 대의 청크를 처리하는

루프를 반복한다. 이러한 병렬 구조는 그림 2.2에 도시된 Time saved 영역만큼의 처리 시간을 단축시킨다. 비록 프로세스 간 통신 및 동기화로 인한 약간의 오버헤드와 프로세스별 처리 속도 차이로 인한 유향 시간이 발생할 수 있으나, 전체 파이프라인의 처리량을 획기적으로 늘려 실시간성을 확보하는 데 결정적인 기여를 한다.

2.3 Pruning 최적화

SpyKING CIRCUS 알고리즘은 실시간으로 변화하는 뇌 신호 환경에 대응하기 위해, 템플릿을 동적으로 생성하고 관리하는 적응형(Adaptive) 학습 과정을 수행한다. 이 과정에서 오랫동안 매칭되지 않거나 중복된 템플릿을 제거하는 가지치기(Pruning) 작업이 빈번하게 발생하는데, 이는 시스템의 메모리 사용량을 일정하게 유지하고 불필요한 비교 연산을 방지하는 필수적인 단계이다. 그러나 수천 개의 템플릿 리스트를 주기적으로 갱신해야 하는 상황에서, 파이썬의 기본 자료구조 처리 방식은 예상치 못한 성능 병목을 유발할 수 있다. 본 절에서는 이러한 문제를 해결하기 위해 적용한 리스트 운용 최적화 기법을 기술한다.

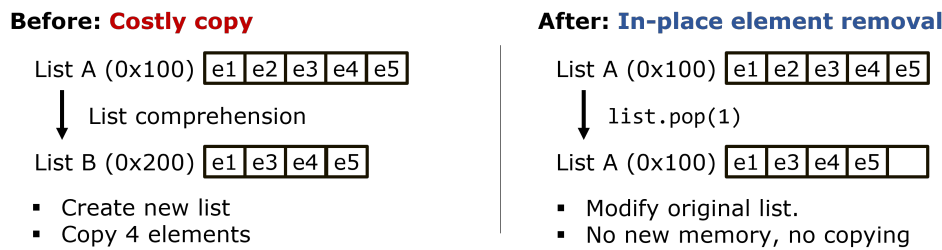


그림 2.3: 메모리 복사 오버헤드 제거를 위한 리스트 운용 구조 개선

Figure 2.3: Improvement of list management structure to eliminate memory copy overhead

기존의 구현에서는 특정 조건을 만족하는 템플릿만을 남기기 위해 파이썬의 리스트 컴프리헨션(List Comprehension) 문법을 사용하였다. 그림 2.3의 좌측 도식(Before: Costly copy)은 이러한 방식의 메모리 동작 과정을 보여준다. 리스트 컴프리헨션은 필터링된 요소들을 담기 위해 별도의 메모리 공간(예: 0x200)에 새로운 리스트 객체를 생성하고, 기존 리스트(0x100)의 데이터를 복사하는 방식으로 동작한다. 비록 파이썬 코드상으로는 간결해 보일지라도, 내부적으로는 매 Pruning 주기마다 $O(N)$ 규모의 메모리 할당과 데이터 이동이 발생하게 된다. 이는 실시간 시스템에서 순간적인 처리 지연을 유발할 뿐만 아니라, 빈번한 메모리 할당과 해제로 인한 가비지 컬렉션(Garbage Collection) 부하를 가중시키는 원인이 된다. 본 연구에서는 이러한 메모리 오버헤드를 제거하기 위해, 그림 2.3의 우측 도식(After: In-place element removal)과 같이 제자리(In-place) 연산 방식으로 로직을 전면 수정하였다. 구체적으로는 새로운 리스트를 생성하는 대신, 기존 리스트 객체(0x100)를 그대로 유지한 채 `list.pop(index)` 메서드를 사용하여 제거 대상인 요소만을 메모리에서 삭제하도록 구현하였다. 이 방식은 불필요한 템플릿을 제거하는 과정에서

새로운 메모리 공간을 할당받을 필요가 없으며, 유지되는 데이터에 대한 복사 연산을 수행하지 않는다. 결과적으로 메모리 대역폭의 낭비를 막고 데이터의 지역성을 유지함으로써, 템플릿의 개수가 많아지는 고부하 환경에서도 안정적인 처리 속도를 확보할 수 있었다.

2.4 메모리 최적화

스파이크 분류 알고리즘의 핵심인 템플릿 매칭 단계에서는 입력된 파형 데이터와 템플릿 간의 유사도를 계산하기 위해 다량의 수치 연산이 수행된다. 그러나 기존 코드의 연산 구조를 정밀 분석한 결과, 메인 루프 내부에서 데이터의 부호를 반전(Negation)시키는 연산이 불필요하게 반복 수행됨으로써 시스템 성능을 저하시키는 요인이 되고 있음을 확인하였다. 본 절에서는 이러한 중복 연산을 제거하고 메모리 접근 효율을 높이기 위해 적용한 최적화 기법을 기술한다.

Before: Redundant data copy	After: Pre-computation
<pre># N = number of data for i=1 to N: # Redundant data copy for negation curr_data = -local_chunk[i] ... (process curr_data) ...</pre>	<pre># One-time negation neg_chunk = -local_chunk for i=1 to N: curr_data = neg_chunk[i] ... (process curr_data) ...</pre>
local_chunk is copied to negate	No local_chunk copy

그림 2.4: 사전 연산 기반의 메모리 접근 구조 개선

Figure 2.4: Improvement of memory access structure based on pre-computation

기존의 구현(Baseline) 방식은 그림 2.4의 좌측 도식(Before: Redundant data copy)과 같이 `curr_data = -local_chunk[curr_idx]` 형태의 연산을 루프 내부에서 수행하였다. 이는 논리적으로는 단순한 부호 반전이지만, 파이썬과 NumPy 환경에서는 반전된 결과를 저장하기 위해 매 반복마다 새로운 임시 메모리 공간을 할당하고 데이터를 복사하는 오버헤드를 유발한다. 수천 번 이상 반복되는 루프 내에서 이러한 메모리 복사가 지속적으로 발생할 경우, CPU의 연산 자원을 낭비할 뿐만 아니라 메모리 대역폭을 비효율적으로 점유하여 전체 처리 속도를 크게 저하시킨다. 이를 개선하기 위해 본 연구에서는 그림 2.4의 우측 도식(After: Pre-computation)과 같이 사전 연산(Pre-computation) 방식을 도입하였다. 루프에 진입하기 전, 전체 데이터 청크에 대해 단 한 번의 부호 반전 연산을 수행하여 `neg_chunk`라는 별도의 배열을 생성한다. 이후 루프 내부에서는 추가적인 연산이나 메모리 할당 없이, 미리 계산된 `neg_chunk`를 단순 슬라이싱(Slicing)하여 사용하도록 구조를 변경하였다. 이 방식은 루프 횟수만큼 반복되던 $O(N)$ 의 메모리 복사 비용을 루프 진입 전 $O(1)$ 회의 연산으로 줄임으로써, 반복적인 메모리 재할당을 원천적으로 차단하고 CPU가 핵심적인 매칭 알고리즘에 집중할 수 있도록 개선하였다.

2.5 연산 최적화

파이썬은 인터프리터 언어의 특성상 반복문을 실행할 때마다 타입 검사 등의 오버헤드가 발생하여, 대규모 수치 연산 시 컴파일 언어에 비해 속도가 현저히 저하되는 단점이 있다. 특히 스파이크 분류와 같이 수천 개의 채널과 템플릿에 대해 동일한 연산을 반복해야 하는 경우, 단순한 `for` 반복문의 사용은 전체 시스템의 심각한 병목이 된다. 본 절에서는 이러한 스칼라 기반의 반복 연산을 NumPy 라이브러리를 활용한 벡터 연산으로 대체하여 CPU 연산 효율을 극대화한 기법을 기술한다.

2.5.1 피크 검출 연산 최적화

스파이크 탐지는 입력된 신호가 설정된 임계값을 넘어서고, 동시에 지역적 극대값인 지점을 찾아내는 과정이다. 기존의 구현은 그림 2.5의 좌측과 같이 채널 개수만큼 반복문을 순회하며, 각 채널의 데이터

Before: Iterative check	After: Vectorized masking
<pre># Check each data point one-by-one for data_point in all_data: if (data_point > threshold) and \ (data_point > neighbors): peaks.append(data_point)</pre>	<pre># Create masks for the entire array mask_A = (all_data > thresholds) mask_B = (all_data > neighbors) # Get all peaks at once peaks = all_data[mask_A & mask_B]</pre>

그림 2.5: 반복문 제거를 통한 병렬적 피크 검출 구조

Figure 2.5: Optimization of peak detection by eliminating loops

포인트에 대해 조건문을 수행하는 방식이었다.

```
if curr_data[i] > thresholds[i] and ...
```

이 방식은 매 반복마다 파이썬 인터프리터가 개입하여 변수의 타입을 확인하고 비교 연산을 수행해야 하므로 막대한 오버헤드가 발생한다.

본 연구에서는 이를 개선하기 위해 그림 2.5의 우측과 같이 불리언 마스킹 기법을 도입하였다. 반복문 없이 전체 채널 데이터 배열(`curr_data_slice`)과 임계값 배열(`thresholds`)을 한 번에 비교하여, 조건을 만족하는 위치를 `True`로 표시하는 마스크 배열을 생성한다.

```
spikes_mask = (curr > th) & (curr > prev) & (curr > next)
```

이후 `numpy.where` 함수를 통해 마스크가 `True`인 인덱스만을 추출함으로써, $O(N)$ 의 파이썬 반복문을 단 한 번의 C 레벨 최적화 연산으로 대체하였다. 실험 결과, 이 최적화를 통해 피크 검출 단계의 소요 시간을 약 10% 단축할 수 있었다.

2.5.2 템플릿 거리 연산 최적화

탐지된 스파이크가 어떤 뉴런에서 발생했는지 식별하기 위해서는, 현재 스파이크 파형과 사전에 학습된 모든 템플릿 파형 간의 유클리드 거리를 계산해야 한다.

Before: Iterative comparison	After: Vectorized calculation
<pre># Find the best match by looping best_dist = 999 for template in all_template: dist = get_distance(spike, template) best_dist = min(dist, best_dist)</pre>	<pre># Calculate all distances at once all_distances = \ get_all_distances(spike, all_templates) # Find the minimum distance instantly best_dist = min(all_distances)</pre>

그림 2.6: 행렬 연산을 통한 다중 템플릿 거리 일괄 계산

Figure 2.6: Optimization of multi-template distance calculation using matrix operations

최적화 이전에는 그림 2.6의 상단 도식처럼 `for` 반복문을 사용하여 템플릿 리스트를 순회하며 개별적으로 거리를 계산하고 최솟값을 갱신하였다.

```
dist = get_distance(spike, template[i])
```

이 경우 템플릿의 개수가 늘어날수록 연산 횟수가 선형적으로 증가하며, 각 거리 계산 함수 호출 시마다 발생하는 스택 오버헤드가 누적된다.

이를 해결하기 위해 본 연구에서는 그림 2.6의 하단과 같이 NumPy의 브로드캐스팅 기능을 활용하여 거리 계산을 벡터화하였다. 입력 스파이크 벡터를 템플릿 행렬 전체와 한 번에 감산 연산한 뒤, `numpy.linalg.norm(..., axis=1)` 함수를 사용하여 모든 템플릿과의 거리를 일괄 계산하였다.

```
distances = norm(all_templates - spike, axis=1)
```

이 방식은 최신 CPU의 SIMD 명령어셋을 효율적으로 활용할 수 있게 하여, 수십 개의 템플릿과의 비교 연산을 병렬적으로 처리한다. 결과적으로 `argmin` 함수를 통해 최소 거리의 인덱스를 즉시 찾아냄으로써 매칭 단계의 처리 지연을 최소화하였다.

제 3 장

결론

3.1 결과 분석

본 연구에서 제안한 최적화 기법의 성능을 검증하기 위해 Intel Xeon E5-2650 v4 프로세서(24 코어)와 261 GiB 메모리를 탑재한 워크스테이션 환경에서 실험을 수행하였다. 실험 데이터로는 64 채널, 30 kHz 샘플링 레이트를 가지는 1200초 분량의 HYBRID_JANELIA 데이터셋을 사용하였으며, 지연 시간(Latency)은 전체 스파이크 분류 과정에 소요된 총 시간으로 정의하였다.

우선, 최적화된 알고리즘의 신뢰성을 보장하기 위해 정확도 검증을 선행하였다. 제안하는 가속 모델을 수행한 결과, 기존 베이스라인 모델과 동일한 개수의 클러스터와 스파이크 할당 결과를 출력함을 확인하였다. 또한 중간 처리 로그를 대조한 결과에서도 수치적 오차가 발생하지 않아, 본 연구의 최적화 과정이 알고리즘의 정확도에 영향을 주지 않으면서 연산 효율만을 개선하였음을 입증하였다.

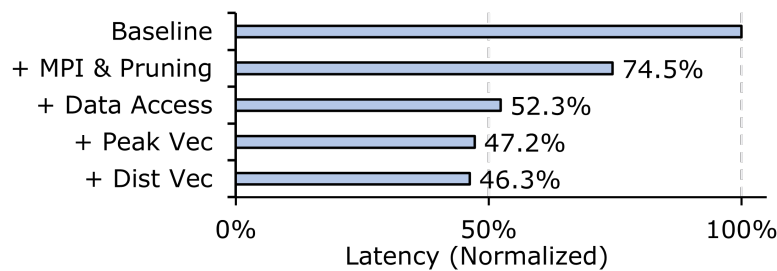


그림 3.1: 최적화 기법의 순차적 적용에 따른 처리 시간 변화

Figure 3.1: Processing time changes according to sequential application of optimization techniques

그림 3.1는 베이스라인 모델에 각 최적화 기법을 단계적으로 적용함에 따른 누적 성능 향상을 보여 준다. 측정 결과, 전체 처리 시간은 최적화 전 516초에서 모든 최적화가 적용된 후 238.7초로 단축되어 총 53.7%의 성능 향상을 달성하였다. 세부적으로 살펴보면, MPI 병렬화와 Pruning 최적화를 적용했을

때 처리 시간이 베이스라인 대비 74.5% 수준으로 감소하여 가장 큰 폭의 성능 개선을 보였다. 이후 사전 연산(Pre-computation)을 통한 메모리 접근 최적화를 적용하자 처리 시간은 52.3% 수준으로 더욱 단축되었다. 마지막으로 피크 검출 및 템플릿 거리 연산에 벡터화 기법을 적용함으로써 최종적으로 46.3% 수준까지 지연 시간을 최소화하였다. 이는 병렬 처리를 통한 구조적 개선뿐만 아니라, 메모리 및 연산 레벨의 미세 최적화가 실시간성 확보에 필수적임을 시사한다.

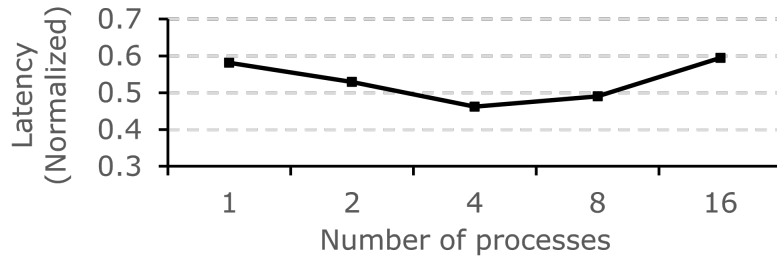


그림 3.2: 병렬화에 따른 처리 시간 변화

Figure 3.2: Processing time changes according to parallelization

그림 3.2는 프로세스 개수 증가에 따른 MPI 병렬 처리의 확장성(Scalability) 분석 결과를 나타낸다. 실험 결과, 단일 프로세스 대비 4개의 프로세스를 사용했을 때 가장 높은 효율(Peak Efficiency)을 보이며 지연 시간이 최소화됨을 확인하였다. 그러나 프로세스 개수를 8개, 16개로 증가시킬 경우 오히려 지연 시간이 다시 증가하는 경향이 관찰되었다. 이는 64채널 데이터셋의 규모에 비해 과도하게 많은 프로세스를 할당할 경우, 각 프로세스가 수행하는 실질적인 연산량은 줄어드는 반면 프로세스 간 통신 및 배리어(Barrier) 동기화에 소요되는 오버헤드가 더 크게 증가하기 때문이다. 따라서 제한된 채널 환경에서는 무조건적인 병렬 확장보다는 데이터 규모에 맞는 적정 프로세스 수를 선정하는 것이 중요함을 알 수 있다.

3.2 기여 및 한계

본 연구는 고성능 신경 신호 처리를 위해 오프라인 기반의 SpyKING CIRCUS 알고리즘을 실시간 처리가 가능한 구조로 개선하고, 다각도의 최적화를 통해 그 효율성을 입증하였다. 본 연구의 주요 기여는 다음과 같이 요약할 수 있다.

첫째, 알고리즘의 정확도를 유지하면서 처리 속도를 획기적으로 개선하였다. 기존의 근사 알고리즘들은 속도를 높이기 위해 정확도를 일부 희생하는 경향이 있었으나, 본 연구는 MPI 병렬화와 벡터화 기법을 통해 원본 알고리즘의 정교한 분류 성능을 100% 유지하면서도 전체 지연 시간(Latency)을 53.7% 단축하는 성과를 거두었다.

둘째, 하드웨어 가속기 없이 소프트웨어 수준에서의 실시간성 확보 가능성을 제시하였다. FPGA나 ASIC과 같은 고가의 전용 하드웨어 없이, 범용 프로세서 환경에서 파이썬 코드의 리팩토링과 병렬 프

로그래밍만으로도 실시간 BCI 시스템의 요구사항에 근접한 처리 속도를 확보할 수 있음을 보였다. 이는 연구 및 개발 단계에서의 비용 효율성과 시스템 유연성을 크게 높여준다.

그러나 본 연구는 다음과 같은 한계점을 지니며, 이는 추후 개선되어야 할 과제로 남는다.

첫째, 통신 오버헤드로 인한 확장성(Scalability)의 제한이다. 실험 결과, 프로세스 개수가 일정 수준(4개)을 초과할 경우 프로세스 간 데이터 교환과 배리어 동기화(Barrier Synchronization)에 소요되는 시간이 연산 이득을 상쇄하여 오히려 전체 지연 시간이 증가하는 현상이 관찰되었다. 이는 채널 수가 제한된 환경에서 무한정 병렬화를 적용하기 어렵다는 것을 의미하며, 데이터 규모에 따른 최적의 자원 할당 전략이 필요함을 시사한다.

둘째, 파이썬 언어 기반의 근본적인 성능 한계이다. 벡터화를 통해 연산 효율을 높였음에도 불구하고, 인터프리터 언어 특유의 실행 오버헤드와 글로벌 인터프리터 락(GIL) 문제는 여전히 존재한다. 전력 제약이 심한 임베디드 BCI 장치에 탑재하기 위해서는 더 낮은 수준(Low-level)의 최적화가 요구된다.

3.3 향후 연구 방향

본 연구의 성과를 발전시키고 실제 임상 및 BCI 응용 분야에 적용하기 위해, 향후 연구는 다음의 세 가지 방향성을 제시한다.

첫째, 다양한 데이터셋에 대한 광범위한 정확도 검증이다. 현재의 HYBRID-JANELLIA 데이터셋뿐만 아니라, 다양한 노이즈 레벨과 발화율을 가진 생체 내(In-vivo) 실제 기록 데이터를 활용하여 최적화된 알고리즘의 강건성을 검증해야 한다.

둘째, 저전력 환경을 위한 C/C++ 또는 CUDA 포팅이다. 현재의 파이썬 구현체를 C/C++로 이식하거나 GPU 가속을 위한 CUDA로 변환함으로써, 전력 소모를 줄이고 처리 속도를 극대화해야 한다. 이는 배터리로 동작하는 이식형(Implantable) 장치나 엣지 컴퓨팅 환경에서의 활용을 위해 필수적이다.

셋째, 실시간 Closed-Loop BCI 시스템으로의 통합이다. 제안한 스파이크 분류 모듈을 실제 신경 자극기나 로봇 의수 제어 시스템과 연동하여, 사용자의 신경 신호를 분석하고 즉각적인 피드백을 제공하는 완전한 형태의 실시간 시스템을 구축하는 것을 최종 목표로 한다.

참고 문헌

- [1] L. R. Hochberg, D. Bacher, B. Jarosiewicz, N. Y. Masse, J. D. Simeral, J. Vogel, S. Haddadin, J. Liu, S. S. Cash, P. van der Smagt *et al.*, “Reach and grasp by people with tetraplegia using a neurally controlled robotic arm,” *Nature*, vol. 485, no. 7398, pp. 372–377, 2012.
- [2] M. S. Lewicki, “A review of methods for spike sorting: the detection and classification of neural action potentials,” *Network: Computation in Neural Systems*, vol. 9, no. 4, pp. R53–R78, 1998.
- [3] P. Yger, G. L. Spampinato, E. Esposito, B. Lefebvre, S. Deny, C. Gardella, M. Stimberg, F. Jetter, G. Zeck, S. Picaud, J. Duebel, and O. Marre, “A spike sorting toolbox for up to thousands of electrodes validated with ground truth recordings in vitro and in vivo,” *eLife*, vol. 7, p. e34518, mar 2018. [Online]. Available: <https://doi.org/10.7554/eLife.34518>
- [4] S. Kim, P. Tathireddy, R. Normann, and F. Solzbacher, “Thermal impact of an active 3-d micro-electrode array implanted in the brain,” *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 15, no. 4, pp. 493–501, 2007.
- [5] K. Sriram, I. Karageorgos, N. Lindsay, M. Wu, R. Opperman, J. Rogers, P. Bose, J. Rajendran, R. Manohar, and A. Bhattacharjee, “Halo: A hardware-software co-designed processor for brain-computer interfaces,” *IEEE Micro*, vol. 43, no. 3, pp. 24–32, 2023.
- [6] K. Sriram, R. P. Pothukuchi, M. Gerasimiuk, M. Ugur, O. Ye, R. Manohar, A. Khandelwal, and A. Bhattacharjee, “Scalo: An accelerator-rich distributed system for scalable brain-computer interfacing,” in *Proceedings of the 50th annual international symposium on computer architecture*, 2023, pp. 1–20.
- [7] U. Rutishauser, E. M. Schuman, and A. N. Mamelak, “Online detection and sorting of extracellularly recorded action potentials in human medial temporal lobe recordings,” *Journal of Neuroscience Methods*, vol. 154, no. 1-2, pp. 204–224, 2006.

Abstract

This study addresses the processing latency and memory constraints of spike sorting algorithms, which are essential for real-time Brain-Computer Interface (BCI) applications. Since multichannel neural signals consist of superimposed firings from multiple neurons, spike sorting is required to identify the firing timing and origin of individual neurons. In particular, closed-loop BCI systems require immediate feedback, necessitating online algorithms that process data upon acquisition under the limited computational and memory resources of embedded environments.

In this paper, we improved the real-time performance of SpyKING CIRCUS, an template-matching-based spike sorting algorithm, by applying parallelization and code-level optimizations. Specifically, channel-based distributed processing was implemented using `mpi4py`, a Python-based parallel communication library. Furthermore, iterative computations were vectorized to eliminate loops, redundant memory copies within loops were removed, and data structures for cluster management were optimized to reduce memory allocation overhead.

The performance was evaluated using the SpikeForest hybrid dataset (30 kHz sampling rate, 64 channels). The total processing time was reduced from 516.0 seconds in the baseline to 238.7 seconds after applying the proposed parallelization and optimizations, achieving a 53.7% improvement. Moreover, the number of clusters and assigned spikes remained identical to the baseline, confirming that sorting accuracy was preserved despite the significant speedup.

The contributions of this study are twofold: first, identifying specific computational bottlenecks through precise profiling and presenting practical solutions to eliminate overheads in loops, memory access, and data structures; second, experimentally demonstrating that implementation-level optimizations can significantly enhance online spike sorting performance and quantitatively analyzing the impact of communication costs on parallel scalability. Although performance inversion due to communication overhead was observed in some parallelization configurations, this study meaningfully enhances the real-time processing capability of online spike sorting while preserving the original algorithmic logic. This work is expected to contribute to increasing the responsiveness of real-time BCI systems and expanding their applicability in resource-constrained environments.

Keywords: Brain-computer interface, Spike Sorting, Parallel Computing, Real-time Processing